

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```

import java.util.Scanner;

public class SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String paragraph = sc.nextLine();

        StringBuffer editHistory = new StringBuffer();
        editHistory.append("---- Editing History ----\n");
        editHistory.append("Original Document: ").append(paragraph).append("\n");

        int charCount = paragraph.length();
        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;

        String[] sentences = paragraph.split("[.!?]");
        int sentenceCount = 0;
        for (String s : sentences) {
            if (!s.trim().isEmpty()) {
                sentenceCount++;
            }
        }

        editHistory.append("Character Count: ").append(charCount).append("\n");
        editHistory.append("Word Count: ").append(wordCount).append("\n");
        editHistory.append("Sentence Count: ").append(sentenceCount).append("\n");
        System.out.println("Enter the word to replace:");
        String oldWord = sc.nextLine();
        System.out.println("Enter the new word:");
        String newWord = sc.nextLine();

        String replacedText = paragraph.replaceAll("\\b" + oldWord + "\\b", newWord);
        editHistory.append("Replaced '").append(oldWord).append("' with '")
            .append(newWord).append("'\n");
        editHistory.append("Text after replacement: ").append(replacedText).append("\n");
        String[] replacedSentences = replacedText.split("[.!?]");
        StringBuilder reversedSentences = new StringBuilder();

        for (String sentence : replacedSentences) {
            String trimmed = sentence.trim();
            if (!trimmed.isEmpty()) {
                String reversed = new StringBuilder(trimmed).reverse().toString();
                reversedSentences.append(reversed).append(". ");
            }
        }

        editHistory.append("Reversed Sentence\n");

        StringBuilder finalDocument = new StringBuilder();
        finalDocument.append("==== Final Formatted Document =====\n");
        finalDocument.append("Character Count: ").append(charCount).append("\n");
        finalDocument.append("Word Count: ").append(wordCount).append("\n");
        finalDocument.append("Sentence Count: ").append(sentenceCount).append("\n");
        finalDocument.append("Text after Replacement: ").append(replacedText).append("\n");
        finalDocument.append("Reversed Sentences: ").append(reversedSentences.toString()).append("\n");

        System.out.println("Original Document");
        System.out.println(paragraph);
    }
}

```

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Integer> id = new ArrayList<Integer>();
        ArrayList<String> name = new ArrayList<String>();
        ArrayList<String> dept = new ArrayList<String>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            id.add(sc.nextInt());
            sc.nextLine();
            name.add(sc.nextLine());
            dept.add(sc.nextLine());
        }

        int updateId = sc.nextInt();
        sc.nextLine();
        String newName = sc.nextLine();
        String newDept = sc.nextLine();

        for (int i = 0; i < id.size(); i++) {
            if (id.get(i) == updateId) {
                name.set(i, newName);
                dept.set(i, newDept);
                break;
            }
        }

        int removeId = sc.nextInt();

        for (int i = 0; i < id.size(); i++) {
            if (id.get(i) == removeId) {
                id.remove(i);
                name.remove(i);
                dept.remove(i);
                break;
            }
        }

        sc.nextLine();
        String searchDept = sc.nextLine();

        System.out.println("Patients:");
        for (int i = 0; i < id.size(); i++) {
            System.out.println(id.get(i) + " " + name.get(i) + " " + dept.get(i));
        }

        System.out.println("Department Patients:");
        for (int i = 0; i < id.size(); i++) {
            if (dept.get(i).equalsIgnoreCase(searchDept)) {
                System.out.println(id.get(i) + " " + name.get(i));
            }
        }

        if (id.size() > 0) {
            System.out.println("First Patient:");
        }
    }
}
```

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Queue<String> queue = new LinkedList<String>();
        PriorityQueue<String> vip = new PriorityQueue<String>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            queue.offer(sc.nextLine());
        }

        int m = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < m; i++) {
            vip.offer(sc.nextLine());
        }

        System.out.println("Current Queue:");
        for (String s : queue) {
            System.out.println(s);
        }

        System.out.println("VIP Queue:");
        for (String s : vip) {
            System.out.println(s);
        }

        if (!queue.isEmpty()) {
            System.out.println("Next Passenger:");
            System.out.println(queue.peek());
        }

        while (!vip.isEmpty()) {
            System.out.println("Boarded VIP: " + vip.poll());
        }

        while (!queue.isEmpty()) {
            System.out.println("Boarded: " + queue.poll());
            System.out.println("Remaining Queue:");
            if (queue.isEmpty()) {
                System.out.println("Empty");
            } else {
                for (String s : queue) {
                    System.out.println(s);
                }
            }
        }

        sc.close();
    }
}

```

INPUT

3
...

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        LinkedHashSet<String> certificates = new LinkedHashSet<String>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            certificates.add(sc.nextLine());
        }

        String search = sc.nextLine();

        if (certificates.contains(search))
            System.out.println("Certificate Found");
        else
            System.out.println("Certificate Not Found");

        System.out.println("Certificates in Insertion Order:");
        for (String s : certificates)
            System.out.println(s);

        System.out.println("Certificates in Sorted Order:");
        TreeSet<String> sorted = new TreeSet<String>(certificates);
        for (String s : sorted)
            System.out.println(s);

        System.out.println("Total Unique Certificates: " + certificates.size());

        String revoke = sc.nextLine();
        certificates.remove(revoke);

        System.out.println("Certificates After Revocation:");
        for (String s : certificates)
            System.out.println(s);

        sc.close();
    }
}
```

INPUT

```
5
C101
C102
C103
C102
C103
C102
C103
```

OUTPUT

(no output)

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashMap<Integer, String> name = new HashMap<Integer, String>();
        HashMap<Integer, Integer> quantity = new HashMap<Integer, Integer>();
        HashMap<Integer, Double> price = new HashMap<Integer, Double>();
        HashMap<Integer, String> category = new HashMap<Integer, String>();

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String pname = sc.next();
            int qty = sc.nextInt();
            double pr = sc.nextDouble();
            String cat = sc.next();

            name.put(id, pname);
            quantity.put(id, qty);
            price.put(id, pr);
            category.put(id, cat);
        }

        int updateId = sc.nextInt();
        int newQty = sc.nextInt();

        if (quantity.containsKey(updateId)) {
            quantity.put(updateId, newQty);
        }

        int searchId = sc.nextInt();

        if (name.containsKey(searchId)) {
            System.out.println("Product Found");
            System.out.println(searchId + " " + name.get(searchId) + " " + quantity.get(searchId) + " " + price.get(searchId) + " " + category.get(searchId));
        } else {
            System.out.println("Product Not Found");
        }

        double total = 0;

        for (Integer id : name.keySet()) {
            total += quantity.get(id) * price.get(id);
        }

        System.out.println("Total Inventory Value: " + total);

        String searchCategory = sc.next();

        System.out.println("Category Products:");
        for (Integer id : name.keySet()) {
            if (category.get(id).equalsIgnoreCase(searchCategory)) {
                System.out.println(id + " " + name.get(id));
            }
        }
    }
}
```